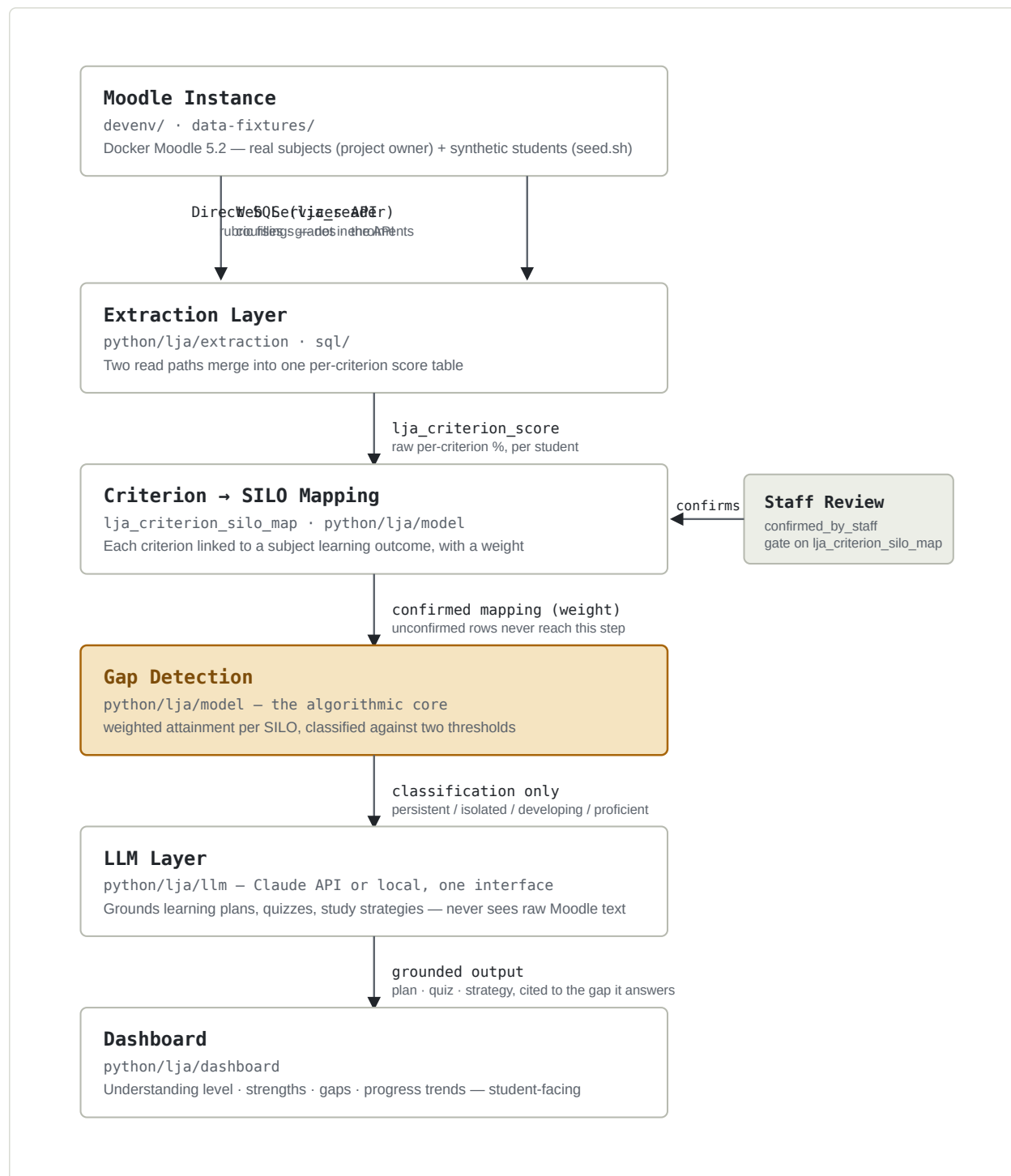


How a Moodle mark becomes a dashboard recommendation

Six stages, one algorithmic core. Everything left of Gap Detection is plumbing; everything right of it is presentation. The gap-classification step is the one piece of real intellectual work in the pipeline — the LLM layer is only ever allowed to see its output, never raw Moodle text.



Top to bottom: a student's mark enters through two extraction paths (the Moodle API doesn't expose rubric fillings, so those are read directly from Postgres), gets mapped to a subject learning outcome only once staff confirm the mapping, is classified into a gap severity, and only that classification — never the underlying Moodle text — is handed to the LLM layer that drafts the plan the dashboard shows.

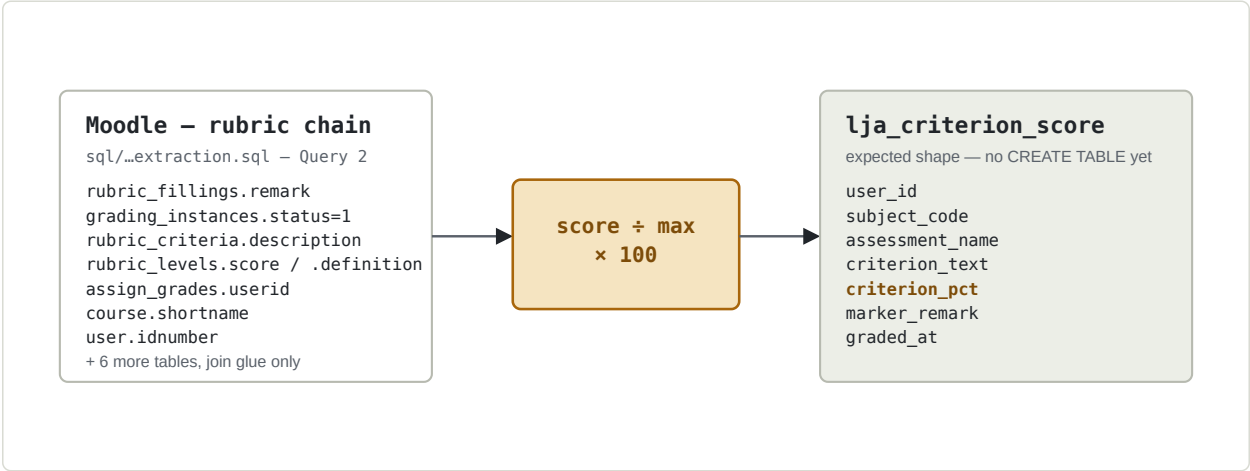
STAGE DETAIL

Owning bundle and build status per stage, as of this sprint.

STAGE	OWNING BUNDLE	INPUT	OUTPUT	STATUS
Moodle Instance	devenv/ , data-fixtures/	— provisions the substrate	Real + synthetic Moodle data	built
Extraction	sql/ (queries) + python/lja/extraction	Moodle Web Services API + read-only Postgres	lja_criterion_score rows	partial — queries written, loader not coded
Mapping	python/lja/model	Criterion scores + staff confirmation	lja_criterion_silo_map (confirmed only)	partial — schema only, no loader
Gap Detection	python/lja/model	Confirmed mapping × scores	Per-SILO attainment % + classification	partial — SQL drafted, not yet Python
LLM Layer	python/lja/llm	Gap classification only	Grounded plan / quiz / strategy	planned
Dashboard	python/lja/dashboard	All of the above	Student-facing views	planned

ZOOM – WHERE LJA_CRITERION_SCORE ACTUALLY COMES FROM

Not an even split. Only the direct-SQL path can reach a rubric *filling* — the level a marker actually chose, and their remark. The Web Services API can hand you the rubric's *definition* (the criteria and levels as configured) but has no function that returns what got awarded. That's the whole reason the SQL bundle exists.



criterion_pct exists because criteria carry different maxima across rubrics and subjects — a "structure" criterion might max at 4 points, "originality" at 8. Normalizing to a percentage first is what makes the weighted SUM(criterion_pct × weight) in Query 6 mathematically meaningful once scores from many different criteria get combined into one SILO attainment number. lja_criterion_score is a name, not a table — Query 6's own comment calls it the "expected upstream table" a loader still has to populate.

Every Moodle field Query 2 actually reads, and what it becomes.

MOODLE FIELD	WHAT IT HOLDS	BECOMES
gradingform_rubric_fillings.remark	Marker's free-text comment on this one criterion	marker_remark
grading_instances.status	Must equal 1 (ACTIVE) — 0/2/3 are stale, draft, or superseded	filter only

grading_instances.itemid	NOT a user id — joins to assign_grades.id , the most common wrong-join in this schema	join key only
grading_instances.raterid	The marker's own user id	grader_user_id
gradingform_rubric_criteria.description	The criterion's own text	criterion_text
gradingform_rubric_levels.score	Points awarded for the level the marker picked	numerator of criterion_pct
gradingform_rubric_levels.definition	That level's descriptor text	level_awarded
MAX(rubric_levels.score) per criterion	The criterion's ceiling (computed, not stored)	denominator of criterion_pct
assign_grades.userid	The student	user_id
assign_grades.grade	Overall gradebook grade for the whole submission	gradebook_grade — a cross-check, not part of criterion_pct
assign.name	The assessment's name	assessment_name
course.shortname	The subject/class code	subject_code
user.idnumber	Student ID number	student_id_number

What the Web Services API gives you instead — roster and course discovery, not criterion-level scores.

FUNCTION	FIELDS	USED FOR
core_webservice_get_site_info	sitename, release, userid, functions[]	Token / connectivity check
core_course_get_courses	id, shortname, fullname	Subject/class-code discovery
core_enrol_get_enrolled_users	id, fullname, email, roles[]	Student roster per subject
gradereport_user_get_grade_items	itemname, graderaw, feedback	Overall (not per-criterion) mark + feedback
core_grading_get_definitions	criteria, levels — same shape as Query 1	Rubric definitions — genuinely redundant with Query 1
mod_assign_get_assignments / mod_assign_get_grades	assignment metadata, grades	Documented, not yet called in moodle_probe.py

No function in this list, or anywhere in Moodle's Web Services API, returns a rubric *filling*. That is the coverage gap python/README.md already documents — Query 2 is not a preference, it's the only path that exists.

Diagram reflects the target architecture from the root README and docs/sprint-plan.md, not what's running today — only the Moodle instance and the SQL/probe spikes exist as code right now. The walking skeleton (Sprint 2) is the point at which every arrow above becomes real.